

Satisfiability

Boolean satisfiability (SAT) is a decision problem that asks whether or not a Boolean formula (i.e., a formula of propositional logic) has a model. That is, is there a way of assigning the truth values {T, F} to the variables in the formula such that the formula evaluates to true?

Boolean satisfiability was the first NP-complete problem. When a problem is complete for a complexity class like NP, it means it is a member of that class, and all problems in that class can be reduced to it. In particular, all problems in the class NP can be formulated as instances of SAT.

The subject of this lecture is local search algorithms for solving SAT. These algorithms are very powerful because they can be used to solve any problem in the class NP via a reduction to SAT (TSP, knapsack, etc.).

1 SAT as Search

Consider the following Boolean formula:

$$\phi \doteq ((A \rightarrow B) \wedge (\neg A \vee B)) \vee ((B \leftrightarrow C) \wedge C)$$

One approach to searching for a satisfying assignment is via truth tables (see Table 1). The trouble with this approach is that there are exponentially many possible assignments: specifically, 2^n , where n is the number of variables in the formula.¹ For our sample formula ϕ , $n = 3$; indeed, there are eight rows in ϕ 's truth table. Of these, six are satisfying assignments, as indicated by the fact that six of the eight rows evaluate to T.

A	B	C	$A \rightarrow B$	$\neg A \vee B$	$(A \rightarrow B) \wedge (\neg A \vee B)$	$B \leftrightarrow C$	$(B \leftrightarrow C) \wedge C$	ϕ
T	T	T	T	T	T	T	T	T
T	T	F	T	T	T	F	F	T
T	F	T	F	F	F	F	F	F
T	F	F	F	F	F	T	F	F
F	T	T	T	T	T	T	T	T
F	T	F	T	T	T	F	F	T
F	F	T	T	T	T	F	F	T
F	F	F	T	T	T	T	F	T

Table 1: A truth table for ϕ . Observe that six of the eight assignments evaluate to T.

Alternatively, we can try to find a satisfying assignment for ϕ using search (see Figure 1). The start state is the empty assignment. The successor correspondence appends an assignment of T or F to the next unassigned variable. The goal states are those in which ϕ evaluates to T.

Even a partial assignment can be a satisfying one: e.g., $A = F$ satisfies ϕ . So an advantage of search is that it can terminate as soon as a satisfying assignment is found. In the worst case, however, when a formula is unsatisfiable, search mimics the truth table method, and is similarly exponential.

¹Professor Eric Ewing, a past instructor for this course, reports that while doing his Ph.D. research, he reduced multiagent path finding to SAT, thereby creating and solving instances of SAT with 10^7 variables.

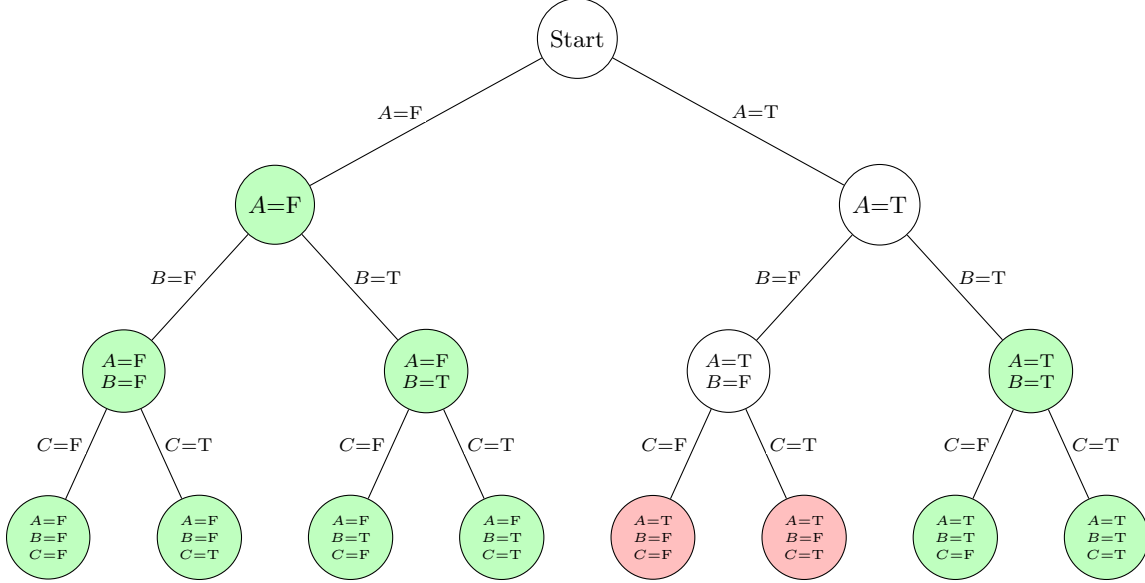


Figure 1: An assignment search tree for ϕ , considering all possible assignments of A , B , and C . Nodes shaded green are satisfying assignments; nodes shaded red are not. Note that even partial assignments (i.e., $A = F$ and $A = T$, $B = T$) can be sufficient for satisfaction.

2 SAT as Optimization

SAT is a decision problem: it asks a yes/no question. Local search algorithms solve optimization problems. So to use local search to solve SAT, we must convert SAT into an optimization problem: i.e., we must define a state space and an objective. Then, to apply local search, we must define a neighborhood operation.

A natural state space for SAT is the space of all assignments of variables to truth values. Assuming variables are ordered in some way, such states can be represented as bit strings: e.g., for $n = 3$ variables, 000, 001, 010, 011, etc. More generally, a bit string $\mathbf{v} \in \{0, 1\}^n$, where v_i denotes the truth value of the i th variable.

Assuming this state representation, a natural neighborhood operation is “flip one bit:” given a state $\mathbf{v}$,

$$\mathcal{N}(\mathbf{v}) = \{\mathbf{u} \in \{0, 1\}^n \mid \text{Hamming Distance}(\mathbf{u}, \mathbf{v}) = 1\}$$

The states $\mathbf{u}$ whose Hamming Distance from $\mathbf{v}$ is 1 are precisely those that are reachable via exactly one bit flip. The size of a neighborhood under this operation is n .

It remains to define an objective function for SAT. To do so, we introduce conjunctive normal form (CNF).

Informally, a formula in CNF consists of multiple OR clauses joined together by ANDs. For example, the following Boolean formula is in CNF: $(\neg A \vee B \vee C) \wedge (\neg A \vee B)$.

We introduce the following language to formally define CNF:

Variables: The variables in a formula (e.g., A, B, C).

Literal: A variable or its negation (e.g., $A, \neg A, B, C$). Variables decorated with $\neg$ are called **negative literals**, while variables without any decoration are called **positive literals**.

Clause: A collection of literals (e.g., $\{\neg A, B, C\}$ and $\{\neg A, B\}$).

CNF: A conjunction of disjunctions of literals.

With this language in hand, we are ready to define Max-SAT, an optimization analog of SAT. Assuming the input to Max-SAT is in CNF, we define the objective value $\text{Obj}(\mathbf{v})$ of a state $\mathbf{v}$ to be the number of unsatisfied clauses, and we seek to maximize the negative of this value. If a model is discovered in which no clauses are unsatisfied, then the input formula is satisfiable.

Remark Lest you worry that restricting the input to our SAT solvers to CNF limits expressiveness, you need not. CNF is universal! Every Boolean formula can be converted into an equivalent CNF formula. Moreover, there is a systematic way of doing so. De Morgan’s Laws and other logical equivalences² can be applied to eliminate implications, push negations inwards, etc.³ The key step is to distribute $\vee$ over $\wedge$: e.g., $\neg A \vee (B \wedge C)$ is equivalent to $(\neg A \vee B) \wedge (\neg A \vee C)$.

Example Conversion to CNF:

$$\begin{aligned} & (A \rightarrow B) \rightarrow (A \rightarrow C) \\ \text{iff } & \neg(\neg A \vee B) \vee \neg A \vee C \\ \text{iff } & (A \wedge \neg B) \vee \neg A \vee C \\ \text{iff } & (A \vee \neg A \vee C) \wedge (\neg B \vee \neg A \vee C) \end{aligned}$$

3 Local Search SAT Solvers

Some of the most powerful algorithms for solving random instances of satisfiability employ local search to solve Max-SAT. These algorithms move around the state space, mostly following the objective function down (to minimize the number of unsatisfied clauses), but occasionally acting otherwise to avoid local minima.

These algorithms are not complete: they return satisfying assignments when they are found; but they are not guaranteed to find a satisfying assignment, even if one exists. Worse, local search algorithms cannot provide certificates of unsatisfiability, so if a run fails, we do not know if the problem has no solution or if we just didn’t run the solver for long enough. These local search approaches work best if you know there is a solution, and they work even better when there are multiple solutions.

GSAT The GSAT algorithm is depicted in Table 2. “G” stands for “greedy,” as this algorithm moves in a direction that leads to the greatest reduction in the number of unsatisfied clauses, breaking ties randomly.

In an attempt to escape local minima, GSAT also implements the “force-best-move” heuristic, which forces it to accept some move, even if the best move beyond the current state is not in fact an improvement.

Let’s apply GSAT to the following CNF formula:

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee x_3)$$

Assume the start state is 000: i.e., $x_1 = 0, x_2 = 0, x_3 = 0$. Which clauses are satisfied?

$x_1 \vee x_2$	F
$\neg x_1 \vee x_3$	T
$\neg x_2 \vee x_3$	T

So there is 1 unsatisfied clause, namely $x_1 \vee x_2$. Since the objective value is not yet zero, search proceeds in the neighborhood of 000. The states in this neighborhood are:

²See the list of tautologies at the end of the Propositional Logic lecture.

³A straightforward application of these rules can lead to an exponential blow up in the size of the formula. It is, however, possible to generate an equisatisfiable formula whose size is polynomial in the size of the original formula.

- 100, if we flip x_1
- 010, if we flip x_2
- 001, if we flip x_3

Next we ask, how many clauses are unsatisfied at each of these neighboring states?

State	000	100	010	001
$x_1 \vee x_2$	F	T	T	F
$\neg x_1 \vee x_3$	T	F	T	T
$\neg x_2 \vee x_3$	T	T	F	T
Total	1	1	1	1

The value of the objective function is 1 at all three of these neighboring states. GSAT's force-best-move heuristic forces a sideways move in this case, since there is no immediate improvement.

GSAT breaks the tie randomly, so let's assume x_1 is randomly selected as the bit to flip, and the state is now 100. The neighborhood of 100 is:

- 000, if we flip x_1
- 110, if we flip x_2
- 101, if we flip x_3

Once again we ask, how many clauses are unsatisfied at each of these neighboring states?

State	000	100	010	001	110	101
$x_1 \vee x_2$	F	T	T	F	T	T
$\neg x_1 \vee x_3$	T	F	T	T	F	T
$\neg x_2 \vee x_3$	T	T	F	T	F	T
Total	1	1	1	1	2	0

This time we find an assignment, namely 101, where there are no remaining unsatisfied clauses. Indeed, we have found a model, and the formula is satisfiable!

GSAT + Walk A variant of GSAT with random walks is inspired by Papadimitriou's random walk algorithm for 2SAT,⁴ which finds a satisfying assignment with probability 1, after $O(n^2)$ bit flips on average, for any satisfiable 2SAT formula with n variables. Given 2SAT formula ϕ ,

REPEAT

- choose unsatisfied clause $C \in \phi$ at random
- choose variable $x \in C$ at random
- flip the value of x

UNTIL all clauses are satisfied

⁴The k SAT problem considers formulas in CNF, but restricts the number of literals per clause to k . 2SAT can be solved in polynomial time, but 3SAT is NP-complete.

GSAT(ϕ, N, M)	
Inputs	CNF formula ϕ number of restarts N number of trials per restart M
Output	satisfying assignment v or fail

for $i = 1$ to N	
1. initialize random start state: i.e., random assignment v	
(a) for $j = 1$ to M	
i. if v satisfies ϕ , return v	
ii. compute the neighborhood of v , $\mathcal{N}(v)$	
iii. let $v \in \arg \min_{u \in \mathcal{N}(v)} \text{Obj}(u)$	
fail	

Table 2: GSAT: Selman, Levesque, and Mitchell [1992].

GSAT+WALK(ϕ, N, M, p)	
Inputs	CNF formula ϕ number of restarts N number of trials per restart M probability p
Output	satisfying assignment v or fail

for $i = 1$ to N	
1. initialize random start state: i.e., random assignment v	
(a) for $j = 1$ to M	
i. if v satisfies ϕ , return v	
ii. with probability p	
A. choose an unsatisfied clause $C \in \phi$ at random	
B. choose a variable $x \in C$ at random	
C. let $v \leftarrow v$ with x 's bit flipped	
iii. with probability $1 - p$	
A. compute the neighborhood of v , $\mathcal{N}(v)$	
B. let $v \in \arg \min_{u \in \mathcal{N}(v)} \text{Obj}(u)$	
fail	

Table 3: GSAT+WALK [Selman, Kautz, and Cohen, 1994].

The **break-value** of a variable x is defined as the number of clauses that were satisfied by v but break when the value of x is flipped. The **make-value** of a variable x is the opposite: the number of clauses that were *not* previously satisfied but become satisfied when the value of x is flipped.

GSAT adopts state changes based on the net value of a state change: make-value minus break-value. GSAT + WALK acts like GSAT with probability $1 - p$, but otherwise acts like a random walk: it chooses an unsatisfied clause $C \in \phi$ at random and a variable $x \in C$ at random, and then flips x 's bit. (See Table 3.) When GSAT + WALK flips a bit in an unsatisfied clause, it instantly renders C satisfied. Thus, the make-value of all variables in this clause is at least 1.

WALKSAT(ϕ, N, M, p)	
Inputs	CNF formula ϕ number of restarts N number of trials per restart M probability p
Output	satisfying assignment v or fail

for $i = 1$ to N
1. initialize random start state: i.e., random assignment v
(a) for $j = 1$ to M
i. if v satisfies ϕ , return v
ii. choose an unsatisfied clause $C \in \phi$ at random
iii. compute the break values of all variables in C
iv. “freebie:” if a variable in C has break-value 0, flip that variable’s value and continue
v. with probability p
A. choose a variable $x \in C$ at random
vi. with probability $1 - p$
A. choose a variable $x \in C$ of minimal break-value (break ties randomly)
vii. let $v \leftarrow v$ with x ’s bit flipped
fail

Table 4: WALKSAT [Selman, Kautz, and Cohen, 1994].

WalkSAT WALKSAT starts by choosing an unsatisfied clause C uniformly at random. All variables in this clause have make-value at least 1.

If there is a variable in C whose break-value is 0, then WALKSAT considers it a “freebie,” and flips that variable’s bit. This variable is deemed a freebie, because flipping its bit is a strict improvement: make-value at least 1, and break-value 0. First, do no harm.

Otherwise, WALKSAT either flips the value of a randomly chosen variable in C with probability p , or flips the value of a variable in C of minimal break-value with probability $1 - p$. The intent of flipping the value of a variable of minimal break-value is to minimize damage to clauses other than C , and thereby avoid the “whack-a-mole” effect, where the algorithm runs around in circles fixing one clause and breaking others, and then fixing those and breaking the originals. Flipping the value of a randomly chosen variable in C instead may mean flipping the value of a variable with a high break-value. But as usual, taking a step in the “wrong” direction may enable the algorithm to escape local minima.

You will explore the tradeoffs between these various SAT solvers in HW 3.

A NP-completeness

The language of NP-completeness is thrown around all over AI, and computer science more broadly, so it is essential that you understand the meaning of the term.

A **reduction** is a method of transforming one problem, say A , into another, say B . More specifically, we assume a solver for B , and we show how to use that solver to solve A . For example, if we want to solve an instance of TSP, we might express that instance as SAT instead of TSP, and then solve it with a SAT solver. This would be a reduction!

Once again, a reduction f from an arbitrary problem A to a specific problem, say SAT, works as follows:

1. Given an instance x of a problem A
2. Apply the reduction f to x , which yields $f(x)$, an instance of SAT
3. Run a SAT solver on $f(x)$, which outputs z
4. Decompile z into a solution to A

N.B. In theoretical computer science, a solver for B , which might only be a theoretical construct, is called an **oracle**. In AI, we build *actual* solvers, albeit sometimes only very slow ones.

A **decision problem** asks a yes-no question of its input, such as whether a formula, the input, is satisfiable: yes or no? An **optimization problem** asks for a best solution as measured by some objective function, either maximizing or minimizing. The decision and optimization formulations of a problem are actually equivalent from the point of view of complexity.⁵

A **complexity class** is a collection of problems with similar time and memory requirements. The complexity class P contains problems that are solvable in polynomial time. The complexity class NP contains problems that are solvable in *nondeterministic* polynomial time. What this means is that a solution to the problem can be guessed (the nondeterminism) and checked in polynomial time. For example, given a proposed solution $\mathcal{I}$ to an instance ϕ of SAT, it is “easy” to verify whether $\mathcal{I}$ is in fact a model of ϕ or not: i.e., to ask whether $\mathcal{I} \models \phi$. You simply plug the assignment $\mathcal{I}$ into ϕ and check whether the formula is satisfied or not.

On the other hand, it is not necessarily easy to find a satisfying assignment: i.e., to find a model of ϕ . Searching for one is like searching for a needle in a haystack, since the space of possibilities is exponential. Therefore, most computer scientists suspect that the class P is distinct from the class NP. Note that P is a clear subset of NP: if you can find a solution in polynomial time, you can also verify it in polynomial time. But the converse, whether $\text{NP} \subseteq \text{P}$, and hence $\text{P} = \text{NP}$, remains open, and is in fact the most important open problem in computer science.

It would be remarkable if every problem whose solution is quick to verify is also quick to solve!

When A is reducible to B in polynomial time, we write $A \leq_P B$. This notation is meant to suggest that solving B is at least as hard as solving A . Alternatively, B is powerful enough to encode A .

A problem is **hard** for a complexity class if all problems $A_1, A_2, \dots$ in that class can be reduced to it in polynomial time. SAT is NP-hard by the Cook-Levin Theorem: $A_i \leq_P \text{SAT}$, for all $A_1, A_2, \dots \in \text{NP}$. In other words, all problems in NP can be encoded as SAT.

A problem is **complete** for a complexity class if it is a member of that class, and hard for that class. As noted earlier, SAT is in NP because it is easy to check whether an assignment is satisfying or not. Therefore, SAT is NP-complete.

⁵A decision problem (i.e., is there a solution with a value $\leq K$ or $\geq K$?) readily reduces to its optimization form: simply solve the optimization problem, and then use the solution to make a decision.

The other direction relies on binary search. Assume an optimization problem with integer solutions, like our formulation of TSP, with a range of possible values, say $\{L, \dots, R\}$, for $L \leq R$. Now, to solve the optimization problem using a decision problem as an oracle, query the oracle at the midpoint $M = \lfloor (L+R)/2 \rfloor$, asking whether there exists a solution whose value is $\leq M$ —for minimization, let’s say. The answer informs a new range of possibilities: if the answer to the query is yes (resp. no), then the upper (resp. lower) bound R (resp. L) is reset to M ($M+1$), and the oracle is queried again. This process continues until $L = R$, at which point the solution value has been found. The total number of queries to the oracle is no more than $\log_2(V)$, the number of bits in a binary representation of V , where $V = O(R - L)$, making this a polynomial-time reduction.